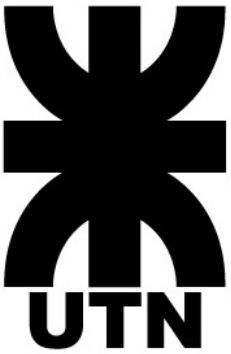
UNIVERSIDAD TECNOLÓGICA NACIONAL

DISEÑO DE SISTEMAS
GRUPO 8

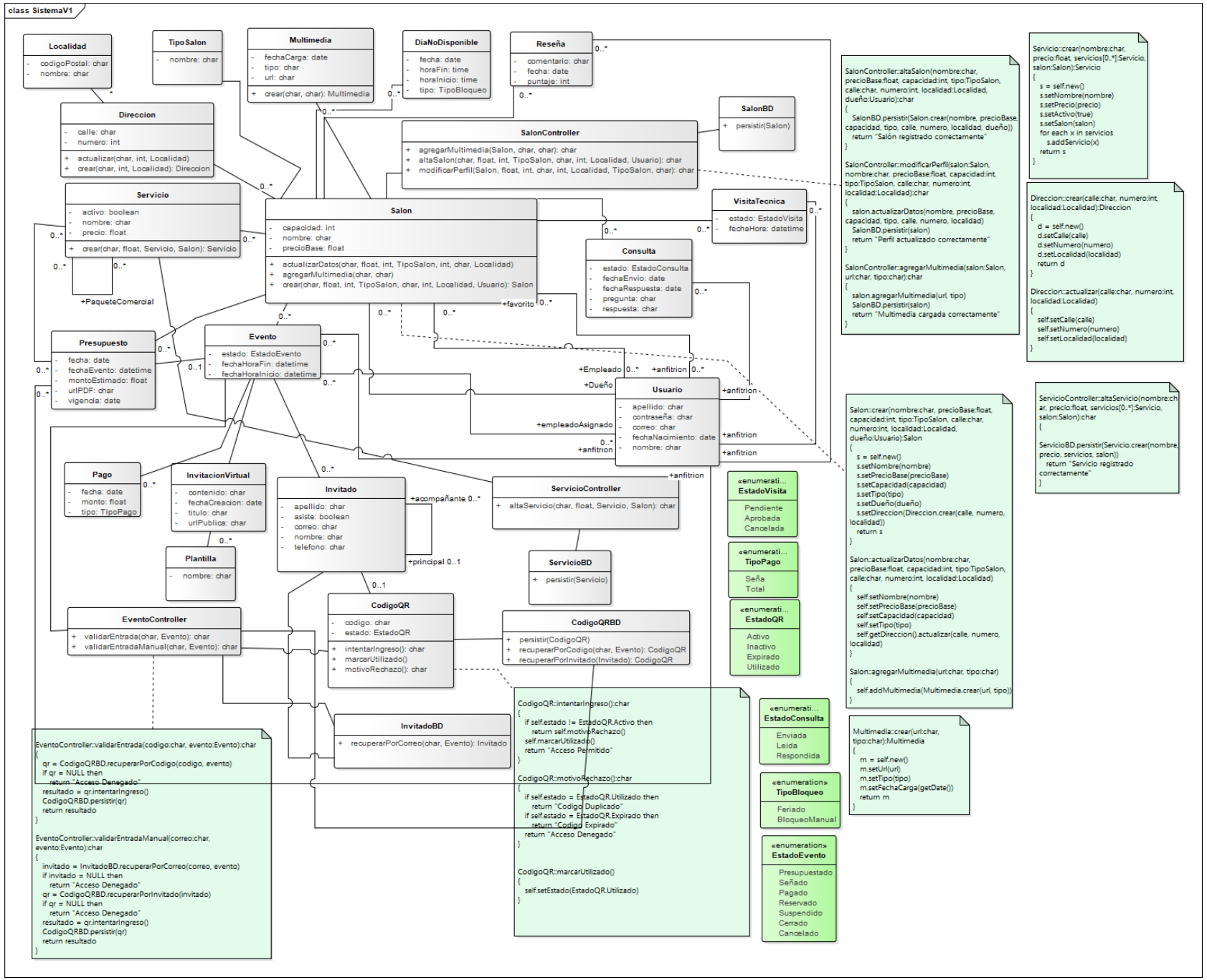


Comisión: 31
Año de cursada: 2026

Integrantes del grupo:

- Mugetti, Emmanuel - Legajo:32534
- Mugetti, Leandro - Legajo: 32535
- Sanchez, Mariano - Legajo: 33079
- Vanucci, Joaquin - Legajo: 33062
- Testa, Daniel - Legajo: 24165

Alcance 3: Eventop


class SistemaV1
Localidad
- codigoPostal: char
- nombre: char
TipoSalon
- nombre: char
Multimedia
- fechaCarga: date
- tipo: char
- url: char
+ crear(char, char): Multimedia
DiaNoDisponible
- fecha: date
- horaFin: time
- horaInicio: time
- tipo: TipoBloqueo
Reseña
- comentario: char
- fecha: date
- puntaje: int
Direccion
- calle: char
- numero: int
+ actualizar(char, int, Localidad)
+ crear(char, int, Localidad): Direccion
SalonBD
+ persistir(Salon)
SalonController
+ agregarMultimedia(Salon, char, char): char
+ altaSalon(char, float, int, TipoSalon, char, int, Localidad, Usuario): char
+ modificarPerfil(Salon, float, int, char, int, Localidad, TipoSalon, char): char
Servicio
- activo: boolean
- nombre: char
- precio: float
+ crear(char, float, Servicio, Salon): Servicio
+PaqueteComercial
Salon
- capacidad: int
- nombre: char
- precioBase: float
+ actualizarDatos(char, float, int, TipoSalon, int, char, Localidad)
+ agregarMultimedia(char, char)
+ crear(char, float, int, TipoSalon, char, int, Localidad, Usuario): Salon
+favorito
VisitaTecnica
- estado: EstadoVisita
- fechaHora: datetime
Consulta
- estado: EstadoConsulta
- fechaEnvio: date
- fechaRespuesta: date
- pregunta: char
- respuesta: char
Presupuesto
- fecha: date
- fechaEvento: datetime
- montoEstimado: float
- urlPDF: char
- vigencia: date
Evento
- estado: EstadoEvento
- fechaHoraFin: datetime
- fechaHoraInicio: datetime
+Empleado
+Dueño
+anfitrion
Usuario
- apellido: char
- contraseña: char
- correo: char
- fechaNacimiento: date
- nombre: char
+empleadoAsignado
Pago
- fecha: date
- monto: float
- tipo: TipoPago
InvitacionVirtual
- contenido: char
- fechaCreacion: date
- titulo: char
- urlPublica: char
Plantilla
- nombre: char
Invitado
- apellido: char
- asiste: boolean
- correo: char
- nombre: char
- telefono: char
+acompañante
+principal 0..1
ServicioController
+ altaServicio(char, float, Servicio, Salon): char
ServicioBD
+ persistir(Servicio)
CodigoQR
- codigo: char
- estado: EstadoQR
+ intentarIngreso(): char
+ marcarUtilizado()
+ motivoRechazo(): char
CodigoQRBD
+ persistir(CodigoQR)
+ recuperarPorCodigo(char, Evento): CodigoQR
+ recuperarPorInvitado(Invitado): CodigoQR
EventoController
+ validarEntrada(char, Evento): char
+ validarEntradaManual(char, Evento): char
InvitadoBD
+ recuperarPorCorreo(char, Evento): Invitado
«enumerati... EstadoVisita
Pendiente
Aprobada
Cancelada
«enumerati... TipoPago
Seña
Total
«enumerati... EstadoQR
Activo
Inactivo
Expirado
Utilizado
«enumerati... EstadoConsulta
Enviada
Leida
Respondida
«enumeration» TipoBloqueo
Feriado
BloqueoManual
«enumeration» EstadoEvento
Presupuestado
Señado
Pagado
Reservado
Suspendido
Cerrado
Cancelado
SalonController::altaSalon(nombre:char, precioBase:float, capacidad:int, tipo:TipoSalon, calle:char, numero:int, localidad:Localidad, dueño:Usuario):char
{
SalonBD.persistir(Salon.crear(nombre, precioBase, capacidad, tipo, calle, numero, localidad, dueño))
return "Salón registrado correctamente"
}
SalonController::modificarPerfil(salon:Salon, nombre:char, precioBase:float, capacidad:int, tipo:TipoSalon, calle:char, numero:int, localidad:Localidad):char
{
salon.actualizarDatos(nombre, precioBase, capacidad, tipo, calle, numero, localidad)
SalonBD.persistir(salon)
return "Perfil actualizado correctamente"
}
SalonController::agregarMultimedia(salon:Salon, url:char, tipo:char):char
{
salon.agregarMultimedia(url, tipo)
SalonBD.persistir(salon)
return "Multimedia cargada correctamente"
}
Servicio::crear(nombre:char, precio:float, servicios[0..*]:Servicio, salon:Salon):Servicio
{
s = self.new()
s.setNombre(nombre)
s.setPrecio(precio)
s.setActivo(true)
s.setSalon(salon)
for each x in servicios
s.addServicio(x)
return s
}
Direccion::crear(calle:char, numero:int, localidad:Localidad):Direccion
{
d = self.new()
d.setCalle(calle)
d.setNumero(numero)
d.setLocalidad(localidad)
return d
}
Direccion::actualizar(calle:char, numero:int, localidad:Localidad)
{
self.setCalle(calle)
self.setNumero(numero)
self.setLocalidad(localidad)
}
Salon::crear(nombre:char, precioBase:float, capacidad:int, tipo:TipoSalon, calle:char, numero:int, localidad:Localidad, dueño:Usuario):Salon
{
s = self.new()
s.setNombre(nombre)
s.setPrecioBase(precioBase)
s.setCapacidad(capacidad)
s.setTipo(tipo)
s.setDueño(dueño)
s.setDireccion(Direccion.crear(calle, numero, localidad))
return s
}
Salon::actualizarDatos(nombre:char, precioBase:float, capacidad:int, tipo:TipoSalon, calle:char, numero:int, localidad:Localidad)
{
self.setNombre(nombre)
self.setPrecioBase(precioBase)
self.setCapacidad(capacidad)
self.setTipo(tipo)
self.getDireccion().actualizar(calle, numero, localidad)
}
Salon::agregarMultimedia(url:char, tipo:char)
{
self.addMultimedia(Multimedia.crear(url, tipo))
}
ServicioController::altaServicio(nombre:char, precio:float, servicios[0..*]:Servicio, salon:Salon):char
{
ServicioBD.persistir(Servicio.crear(nombre, precio, servicios, salon))
return "Servicio registrado correctamente"
}
Multimedia::crear(url:char, tipo:char):Multimedia
{
m = self.new()
m.setUrl(url)
m.setTipo(tipo)
m.setFechaCarga(getDate())
return m
}
CodigoQR::intentarIngreso():char
{
if self.estado != EstadoQR.Activo then
return self.motivoRechazo()
self.marcarUtilizado()
return "Acceso Permitido"
}
CodigoQR::motivoRechazo():char
{
if self.estado = EstadoQR.Utilizado then
return "Codigo Duplicado"
if self.estado = EstadoQR.Expirado then
return "Codigo Expirado"
return "Acceso Denegado"
}
CodigoQR::marcarUtilizado()
{
self.setEstado(EstadoQR.Utilizado)
}
EventoController::validarEntrada(codigo:char, evento:Evento):char
{
qr = CodigoQRBD.recuperarPorCodigo(codigo, evento)
if qr = NULL then
return "Acceso Denegado"
resultado = qr.intentarIngreso()
CodigoQRBD.persistir(qr)
return resultado
}
EventoController::validarEntradaManual(correo:char, evento:Evento):char
{
invitado = InvitadoBD.recuperarPorCorreo(correo, evento)
if invitado = NULL then
return "Acceso Denegado"
qr = CodigoQRBD.recuperarPorInvitado(invitado)
if qr = NULL then
return "Acceso Denegado"
resultado = qr.intentarIngreso()
CodigoQRBD.persistir(qr)
return resultado
}

RF-001 — Gestión de Perfil del Salón

- **Descripción:** Registrar y modificar datos del perfil: tipo, precio base/hora, ubicación, capacidad, disponibilidad y multimedia.
- **Escenario de prueba:** Modificar el precio base y la capacidad del salón, verificar que los cambios se guarden y actualicen inmediatamente.
- **Finalidad:** Permitir al salón exponer su oferta comercial parametrizada en la plataforma.
  Objetivo EvenTop: Registro de salones, Encontrar salones por disponibilidad.

RF-214 — Control de Acceso

- **Descripción:** Validar la entrada de invitados en tiempo real (manual o vía escaneo QR).
- **Escenario de prueba:** Escanear un código QR válido en la entrada del salón y validar que la pantalla devuelva un mensaje verde de "Acceso Permitido".
- **Finalidad:** Profesionalizar el ingreso al evento, erradicando los fraudes y las largas filas en la puerta

---

A continuación resumimos los cambios que hicimos en el diagrama de sistema para responder a cada punto de la devolución.

1. PaqueteComercial y Presupuesto

- Eliminamos la clase `PaqueteComercial`.
- Un paquete ahora es un `Servicio` que agrupa otros servicios. Para eso agregamos una asociación reflexiva en `Servicio` con el rol `+PaqueteComercial`, como en el boceto sugerido.
- Como `Presupuesto` ya se asocia con `Servicio`, un presupuesto puede incluir servicios sueltos o paquetes

2. Registro de los servicios del salón

- Agregamos `ServicioController`, con la operación `altaServicio`, y `ServicioBD`, que guarda el servicio.
- `Servicio::crear` arma el servicio y lo asocia a su salón con `setSalon`. Si recibe otros servicios, los agrupa y el servicio creado es un paquete; con la lista vacía se crea un servicio suelto.

3. Patrón “No hables con extraños” en el control de acceso

- `CodigoQR` ya no recibe `evento`. El controlador lo usa solo para buscar en la BD el código QR de ese evento con `CodigoQRBD.recuperarPorCodigo(codigo, evento)`, igual que `InvitadoBD.recuperarPorCorreo(correo, evento)` en el ingreso manual.
- `intentarIngreso()` no recibe parámetros: `CodigoQR` decide con su propio estado. Si está activo, se marca como utilizado y devuelve “Acceso Permitido”; si no, devuelve el motivo del rechazo.
- Eliminamos `CodigoQR::esValido` e `Invitado::perteneceA`, que dejaron de ser necesarias.
- En el ingreso manual, el QR se obtiene con `CodigoQRBD.recuperarPorInvitado(invitado)` en lugar de navegar `invitado.getCodigoQR()`.